

Bases de données NOSQL et Big DataP

- TP2 : Traitement Batch avec Hadoop Streaming -

Objectifs :

Ce TP vise à vous familiariser avec l'usage d'Hadoop Streaming pour la mise en œuvre de tâches de traitement de données via MapReduce en utilisant Python. Vous apprendrez à développer des programmes de Map et de Reduce en Python, puis à exécuter des job MapReduce avec Hadoop Streaming.

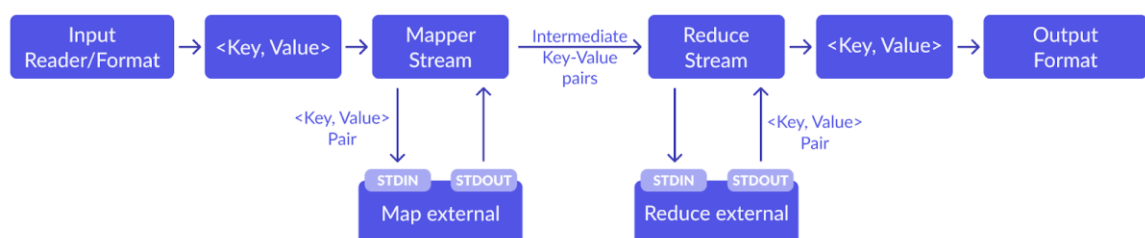
N.B : Commencer par installer python3 sur tous les nœuds du cluster.

Partie 1 : Comprendre Hadoop Streaming

Hadoop Streaming est un outil qui permet d'exécuter des tâches MapReduce avec des programmes personnalisés en utilisant des langages autres que Java (comme Python, Perl, Ruby, etc.). C'est une approche simple dans laquelle Hadoop invoque un exécutable en tant que partie d'une tâche plutôt que d'héberger un fichier JAR dans un environnement Java.

Il fonctionne en prenant des flux d'entrée et de sortie depuis et vers les tâches mappage (mapper) et réduction (reducer) sous forme de lignes textuelles. Les programmes personnalisés lisent depuis l'entrée standard (STDIN) et écrivent vers la sortie standard (STDOUT). Comprendre comment STDIN et STDOUT sont utilisés en Python est essentiel pour utiliser l'API Hadoop Streaming.

Hadoop Streaming



Le programme qui exécute la fonction de map est appelé le **"mapper"**. Le programme qui exécute la fonction de réduction est appelé le **"reducer"**. Les lignes de la sortie standard du processus de map sont converties en paires clé/valeur en les séparant sur le premier caractère de tabulation ('\t', qui est le caractère de séparation par défaut mais peut être modifié). Les paires clé/valeur sont ensuite envoyées à l'entrée standard du reducer pour un traitement ultérieur. Enfin, le reducer écrit sur la sortie standard, qui est la sortie finale du programme.

Partie 2 : Exécution d'un job avec Hadoop Streaming

Dans cet exemple, nous allons voir le code du **WordCount** pour compter combien de fois chaque mot apparaît dans un ensemble de fichiers texte. Le programme se compose d'une fonction de map (mapper) et d'une fonction de reduce (reducer). Initialement, nous allons tester le code localement sur de petits fichiers avant de l'utiliser dans un job de streaming MapReduce. Il est beaucoup plus facile de déboguer les erreurs de codage sur votre ordinateur personnel que sur le cluster.

Mapper : mapperWC.py

Le programme mapper lit chaque ligne d'entrée, découpe les mots et émet un couple clé-valeur pour chaque mot, où la clé est le mot et la valeur est 1. Le programme renvoie (écrit) ces paires clé-valeur sur STDOUT. Un délimiteur (par défaut, "\t") doit être utilisé entre la clé et la valeur pour le traitement correct d'une paire clé-valeur lors de l'étape de Shuffle.

En utilisant votre environnement de développement Python, créez un fichier mapperWC.py. Il est préférable de saisir ce code manuellement plutôt que de le copier/coller pour mieux comprendre ce que fait le code.

```
#!/usr/bin/env python3
"""mapper.py"""
import sys
# input comes from STDIN (standard input)
for line in sys.stdin:
    # remove leading and trailing whitespace
    line = line.strip()
    # split the line into words
    words = line.split()
    # increase counters
    for word in words:
        # write the results to STDOUT (standard output);
        # what we output here will be the input for the reducer.py
        # tab-delimited; the trivial word count is 1
        print('{}\t{}'.format(word, 1))
```

Reducer: reducerWC.py

Le programme reducer agrège les paires clé-valeur en additionnant le nombre de fois que chaque mot apparaît. Il renvoie le résultat sur STDOUT. Créez un fichier reducerWC.py et saisissez le code suivant :

```
#!/usr/bin/env python3
import sys
# Initialize variables
current_word = None
current_count = 0
# Iterate through input lines, which are sorted by key (word) in ascending order
for line in sys.stdin:
    # Remove leading and trailing whitespace
    line = line.strip()
    # Split the key (word) and value (count) by a tab character
    word, count = line.split('\t', 1)
    # Convert the count to an integer
    try:
```

```
        count = int(count)
    except ValueError:
        # If the conversion fails, skip this line
        continue
    # If the current word is the same as the previous word, increment the count
    if current_word == word:
        current_count += count
    else:
        # If the word changes, print the result for the previous word
        if current_word:
            print('{}\t{}'.format(current_word, current_count))
        # Reset the variables for the new word
        current_word = word
        current_count = count
# Print the result for the last word
if current_word is not None:
    print('{}\t{}'.format(current_word, current_count))
```

Exécution du programme en local :

Il est judicieux de tester le code localement sur de petits fichiers avant de l'utiliser dans une tâche de streaming MapReduce. Vous allez effectuer des tests locaux conformément à la méthode de pipeline typique de style UNIX. Pour ce faire, vous allez combiner l'entrée standard (stdin) et la sortie standard (stdout) de votre script Python (mapperWC.py) en utilisant des commandes de redirection de flux. Vous devez avoir un fichier (ici « **input.txt** ») pour tester vos codes.

Ouvrez PowerShell (ou un autre terminal si vous utilisez un système d'exploitation différent) et tapez la commande suivante :

```
cat input.txt |python3 mapperWC.py | sort |python3 reducerWC.py
```

Vous pouvez faire le même test sur votre container namenode.

Exécution du programme sur le cluster :

Après avoir exécuté le code avec succès localement, l'étape suivante consiste à l'exécuter sur le cluster avec Hadoop Streaming. Pour cela, il faut suivre les étapes suivantes :

1. Transférer les scripts dans le cluster Hadoop :

```
docker cp mapper.py namenode:/mapperWC.py
docker cp reducer.py namenode:/reducerWC.py
```

2. Entrer dans le container du namenode et tester les scripts :

Pour se placer dans le container namenode, exécuter la commande suivante :

```
docker exec -it namenode bash
```

Pour rendre les scripts exécutables, exécutez les commandes suivantes:

```
chmod u+x mapperWC.py
chmod u+x reducerWC.py
```

Tester les scripts sur le container (qui est une machine Linux):

```
cat input.txt |python3 mapperWC.py | sort |python3 reducerWC.py
```

3. Créer des fichiers textes et les transférer dans HDFS :

En premier lieu, vous allez tester votre code sur les petits fichiers suivants. Vous pouvez par la suite le tester sur les fichiers du dossier **Data** (créé dans le TP1.)

```
mkdir input
echo "Hello World" > input/f1.txt
echo "Hello Docker" > input/f2.txt
echo "Hello Hadoop" > input/f3.txt
echo "Hello MapReduce" > input/f4.txt
hadoop fs -mkdir -p input
hdfs dfs -put ./input/* input
```

4. Exécuter le programme MapReduce

Pour cela, vous devez d'abord localiser le fichier JAR Hadoop Streaming :

```
find / -name 'hadoop-streaming*.jar'
```

Pour exécuter le programme MapReduce, il faut lancer la commande suivante :

```
hadoop jar /opt/hadoop-3.2.1/share/hadoop/tools/lib/hadoop-streaming-3.2.1.jar \
-files mapperWC.py,reducerWC.py \
-input input \
-output output \
-mapper "python3 mapperWC.py" \
-reducer "python3 reducerWC.py"
```

Avec:

- **input** = Emplacement d'entrée pour le Mapper, à partir duquel il peut lire les données d'entrée.
- **output** = Emplacement de sortie pour le Reducer afin de stocker les résultats. **Il ne doit pas exister. Hadoop va le créer.**
- **mapperWC.py** = Le fichier exécutable du Mapper.
- **reducerWC.py** = Le fichier exécutable du Reducer.
- **files** = spécifie un fichier à emballer avec le job. Cela peut être très utile pour emballer également tout fichier auxiliaire que votre programme pourrait utiliser (dictionnaires, fichiers de configuration, etc).

5. Voir les résultats :

Pour voir les résultats, vous pouvez lister les fichiers de sortie et les afficher :

```
hdfs dfs -ls output hdfs ds -cat output/part-r-00000
```

6. Améliorer les codes :

Apportez les corrections et optimisations nécessaires aux codes du mapper et/ou reducer.